

Lecture 15: Dijkstra's Algorithm

*Instructor: Sourav Chakraborty**Scribe: Kartik Raman, Kaushal Pal*

Problem Statement

Given a graph $G = (V, E, W)$ and two vertices s (source) and t (target), find the minimum distance (shortest path) from s to t , where V is the vertex set, E is the edge set and W is the weight function defined as $W : E \rightarrow \mathbb{R}^+$.

Case 1: Unweighted Graph

If all edges have equal weights (assume $W(e) = 1$ for all $e \in E$), we can use **Breadth-First Search (BFS)**.

Case 2: Weighted Graph (Using Dijkstra)

Why we cannot use BFS?

In BFS, the shortest path is defined by the number of edges (hops). In a weighted graph, it is defined by the sum of edge weights. BFS assumes every edge has a cost of 1, reaching a node for the first time via the fewest hops. However, a path with more edges may have a lower total weight. Therefore, BFS may fail in weighted graphs.

Algorithm for the Shortest Path

- **Initialize:** $dist[v] = \infty$ for all v , $dist[source] = 0$. $Q = V$.
- **Process:** While Q is not empty:
 - Pick $u \in Q$ with smallest $dist[u]$.
 - Remove u from Q .
 - For each neighbor v of u still in Q :
 - * Calculate $alt = dist[u] + W(u, v)$.
 - * If $alt < dist[v]$, update $dist[v] = alt$.

Worked Out Example

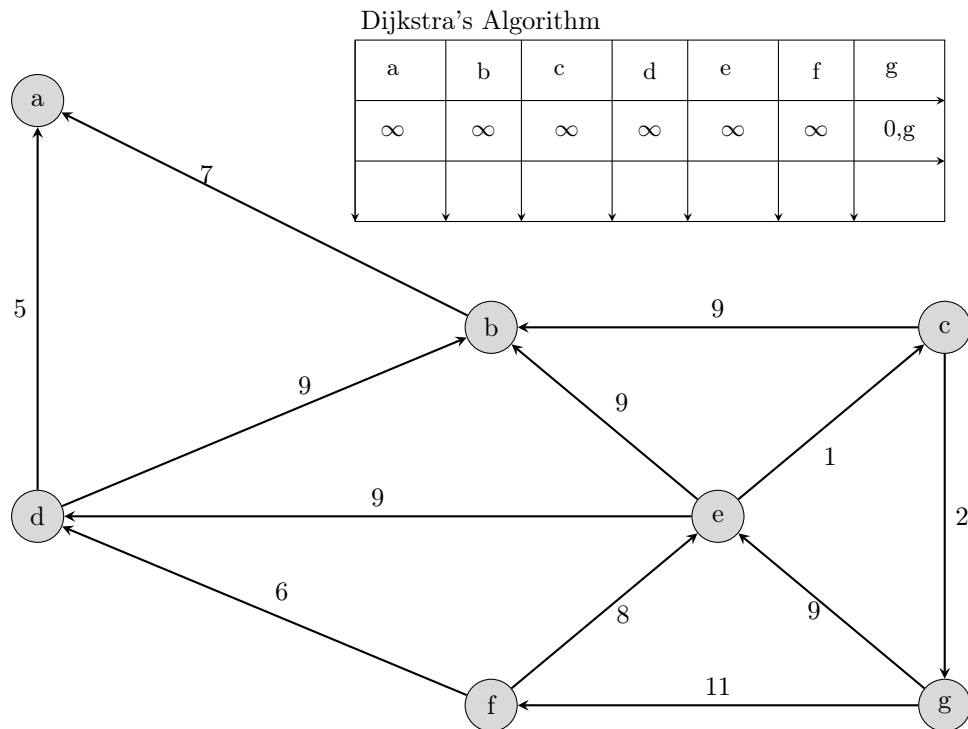


Figure 1: Initial Dijkstra's State

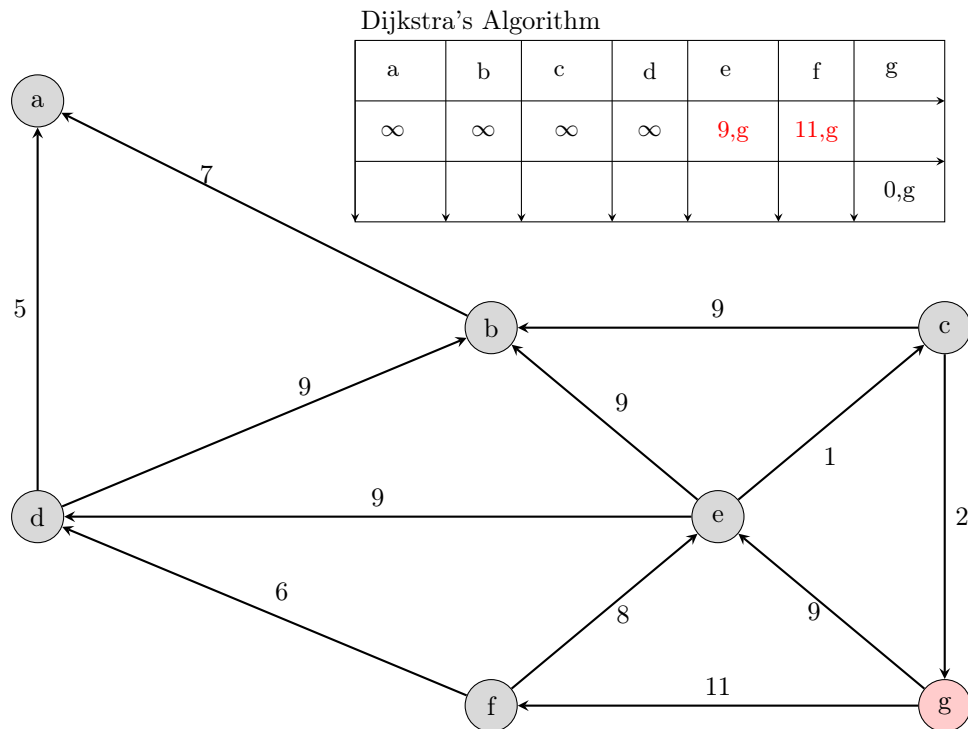


Figure 2: State after source 'g' is active

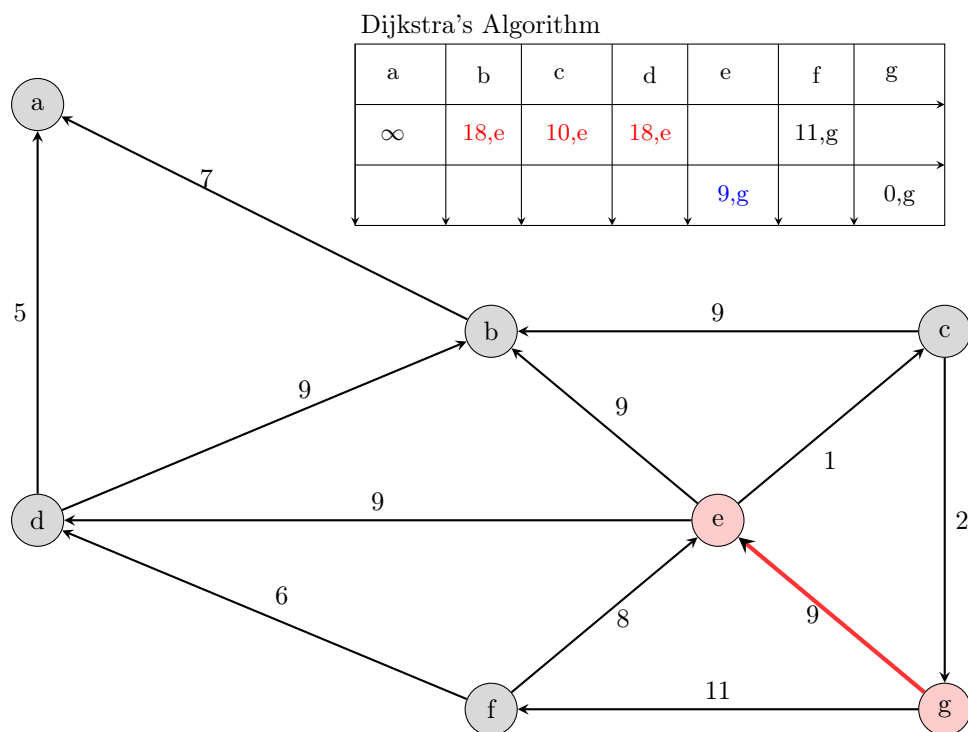


Figure 3: State after relaxing 'g-e'

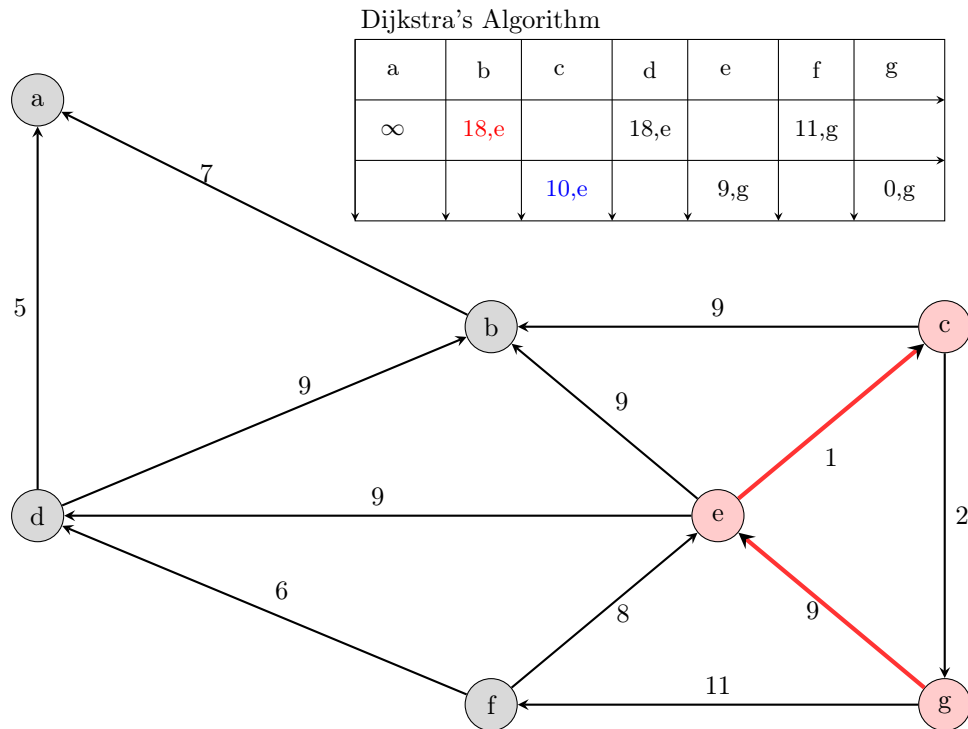


Figure 4: State after relaxing 'e-c'

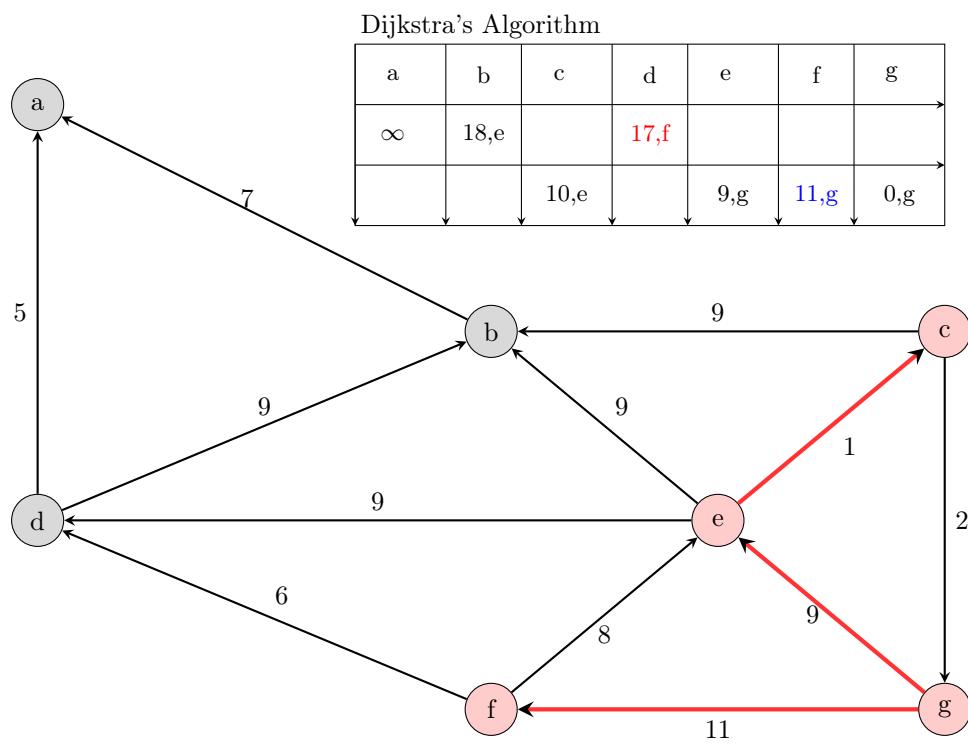


Figure 5: Relaxing neighbouring edges from 'f'

Dijkstra's Algorithm

a	b	c	d	e	f	g
22,d	18,e					
		10,e	17,f	9,g	11,g	0,g

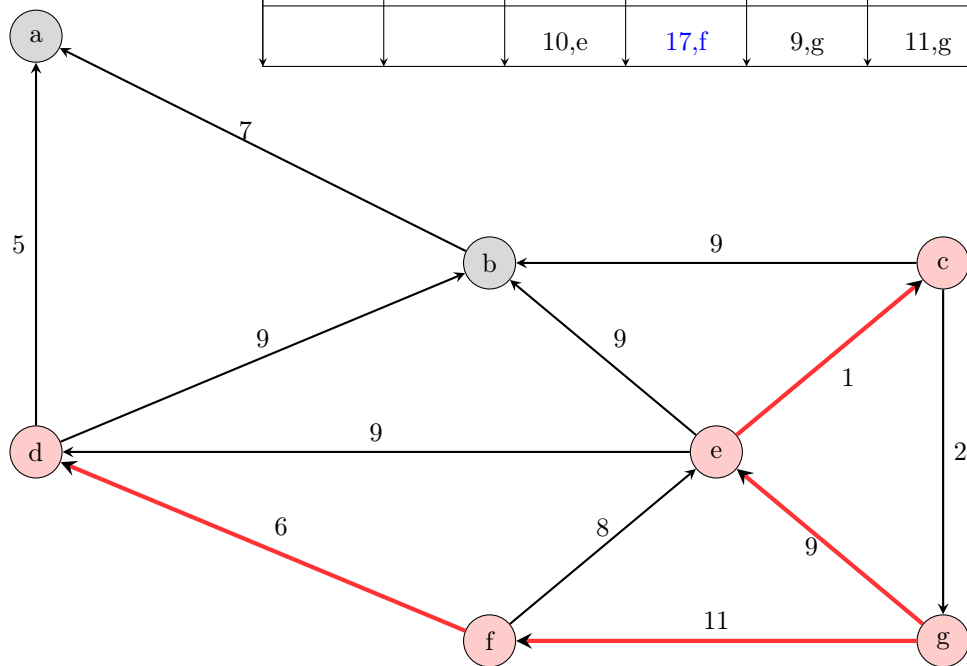


Figure 6: State with multiple finalized active paths

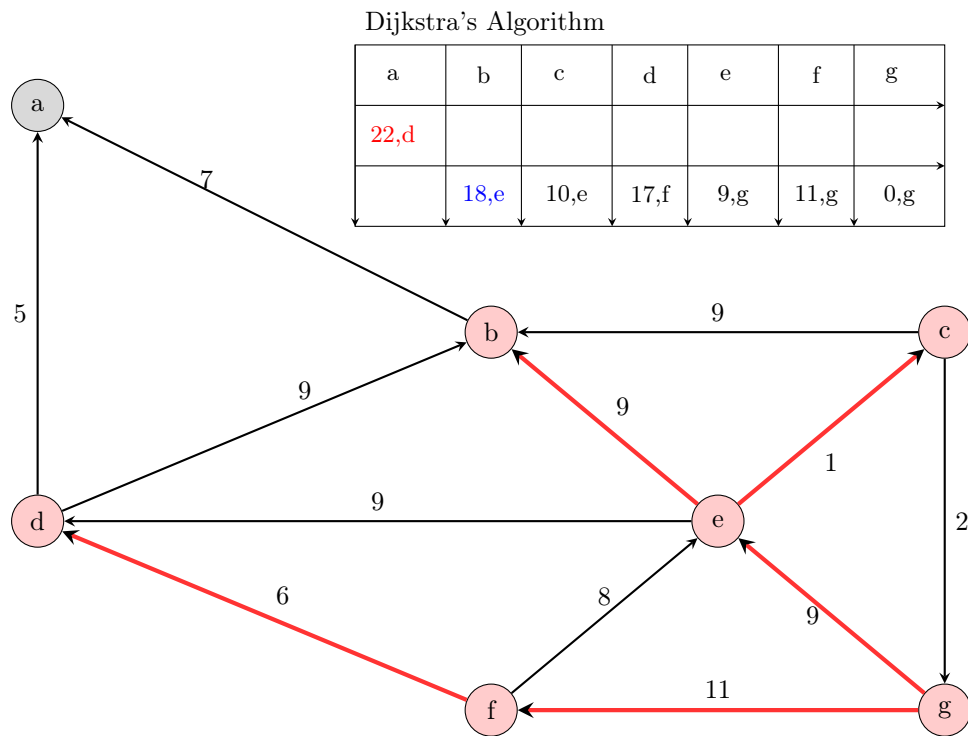


Figure 7: State with final path to 'a' activated

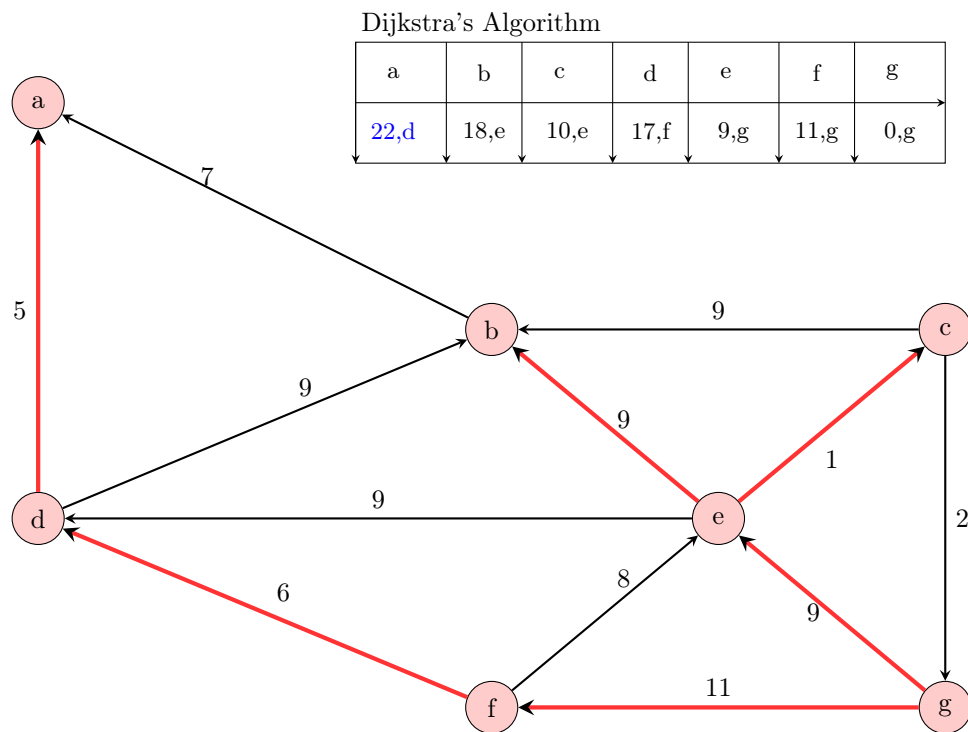


Figure 8: State with finalized path to 'a'

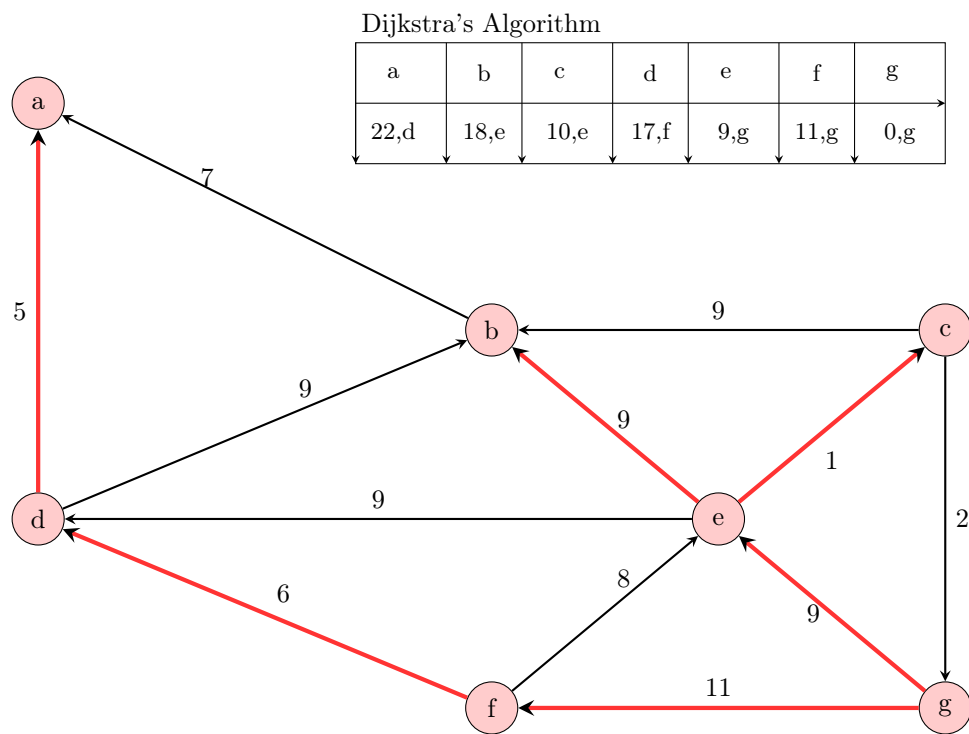


Figure 9: State with finalized path to 'a'

The algorithm used above is known as Dijkstra's Algorithm.

Pseudocode of Dijkstra's Algorithm.

```
for all vertex  $v$  in Graph do
     $dist[v] \leftarrow \infty$ 
end for
 $dist[source] \leftarrow 0$ 
 $Q \leftarrow V$ 
while  $Q \neq \emptyset$  do
     $u \leftarrow$  vertex in  $Q$  with smallest  $dist[u]$ 
    Remove  $u$  from  $Q$ 
    for all neighbor  $v$  of  $u$  do
         $alt \leftarrow dist[u] + W(u, v)$ 
        if  $alt < dist[v]$  then
             $dist[v] \leftarrow alt$ 
        end if
    end for
end while
```

Correctness: Proof of Dijkstra's Algorithm

Notations

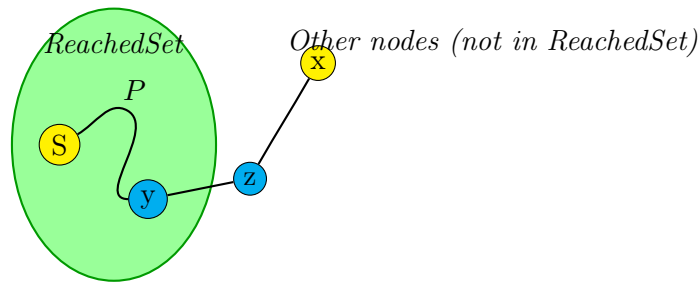
- $D(S, u)$: The **minimum distance** computed by **Dijkstra's algorithm** between nodes S and u .
- $d(S, u)$: The **actual minimum distance** between nodes S and u .

Proposition (Goodrich 13.23). In Dijkstra's Algorithm, when a vertex u is included into the *ReachedSet*, we have:

$$D(S, u) = d(S, u)$$

Proof. Suppose the **statement is false**. Let x be the **first** vertex such that when x is included into the *ReachedSet*, $D(S, x) > d(S, x)$. This implies that for all previous vertices z already in the *ReachedSet*, $D(S, z) = d(S, z)$.

Graphical depiction of the situation:



Let P be the (real) **shortest path** from S to x . Let z be the **first vertex** on P that is **not** in the *ReachedSet*, and let y be its predecessor.

1. $D(S, y) = d(S, y)$ because y was included before x .
2. $D(S, z) = D(S, y) + \text{linkcost}(y, z) = d(S, y) + \text{linkcost}(y, z)$.
3. $D(S, x) \leq D(S, z)$ because the algorithm chose to include x next, meaning x had the smallest current distance estimate.

Important Fact: A sub-path of a shortest path is itself a shortest path. Therefore:

$$d(S, x) = d(S, z) + d(z, x)$$

Conclusion:

$$D(S, x) \leq D(S, z) \quad (\text{From step 3})$$

$$= d(S, y) + \textit{linkcost}(y, z) \quad (\text{From step 2})$$

$$= d(S, z)$$

$$\leq d(S, z) + d(z, x) = d(S, x)$$

This results in $D(S, x) \leq d(S, x)$, which **contradicts** our initial assumption that $D(S, x) > d(S, x)$. Therefore, the statement must be **true**. $\square$

Implementations and Time Complexity Analysis

There are multiple ways to implement Dijkstra's Algorithm. The choice of data structures for both the graph representation and the priority queue directly impacts the final time complexity.

Case 1: Adjacency Matrix and Unordered List

This case occurs when:

- The graph $G = (V, E)$ is represented as an **adjacency matrix** $w[u, v]$.
- The priority queue Q is represented as an **unordered list**.

The calculation is as follows:

1. **Initialization:** Adding all $|V|$ vertices to Q takes $O(|V|)$ time.
2. **Extract-Min:** Searching for the node with the minimal *dist* in an unordered list takes $O(|V|)$. Since we do this for every vertex, the total time is $O(|V|^2)$.
3. **Relaxation:** For each vertex removed, we scan its row in the adjacency matrix ($|V|$ entries) to update distances.

4. **Total Complexity:**

$$O(|V|) + O(|V| \times |V|) = O(|V|^2)$$

Case 2: Adjacency List and Binary/Fibonacci Heap

This case is valid when the graph uses an **adjacency list** and Q uses a more efficient heap structure.

Using a Binary Heap

1. **Building the Heap:** $O(|V|)$ time to construct the initial queue.
2. **Extract-Min:** Removing the node with minimal *dist* takes $O(\log |V|)$. This happens $|V|$ times.
3. **Decrease-Key (Relaxation):** Each edge $|E|$ is relaxed at most once. In a binary heap, updating a value takes $O(\log |V|)$.
4. **Total Complexity:**

$$O(|V|) + O(|E| \log |V|) + O(|V| \log |V|)$$

Since $|E| \geq |V| - 1$ for a connected graph, this simplifies to:

$$O(|E| \log |V|)$$

Using a Fibonacci Heap

A Fibonacci Heap improves the efficiency of the Decrease-Key operation.

- **Extract-Min:** $O(\log |V|)$ amortized time.
- **Decrease-Key:** $O(1)$ amortized time.

The resulting total complexity is:

$$O(|E| + |V| \log |V|)$$

Time Complexity Comparison Summary

Priority Queue	Graph Representation	Time Complexity
Unordered List	Adjacency Matrix	$O(V^2)$
Binary Heap	Adjacency List	$O(E \log V)$
Fibonacci Heap	Adjacency List	$O(E + V \log V)$

Overall, the **Fibonacci heap-based implementation** is theoretically the fastest, especially for sparse graphs. Conversely, the **unordered list** is the slowest, though it performs comparably on very dense graphs where $E \approx V^2$.

What if edge weights are negative?

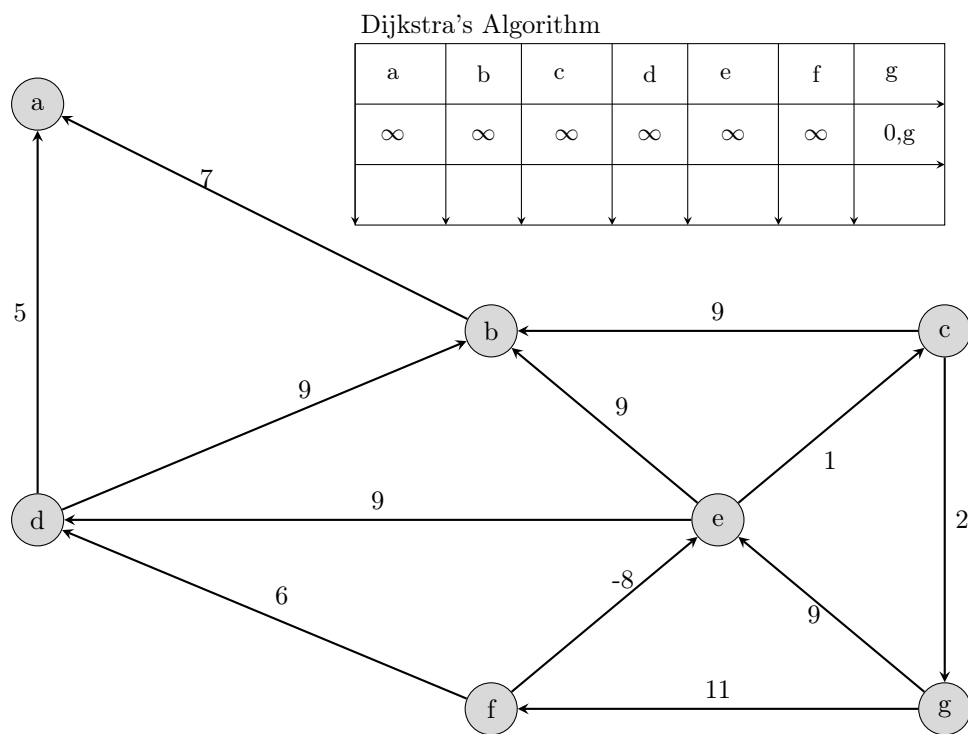


Figure 10: Initial Dijkstra's State

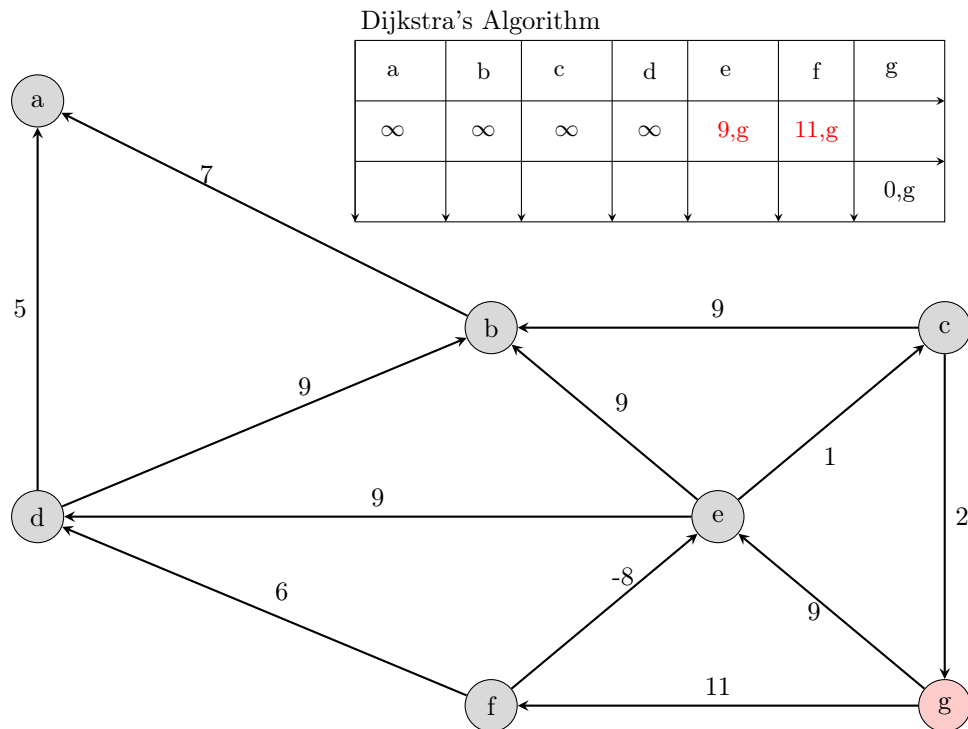


Figure 11: State after source 'g' is active

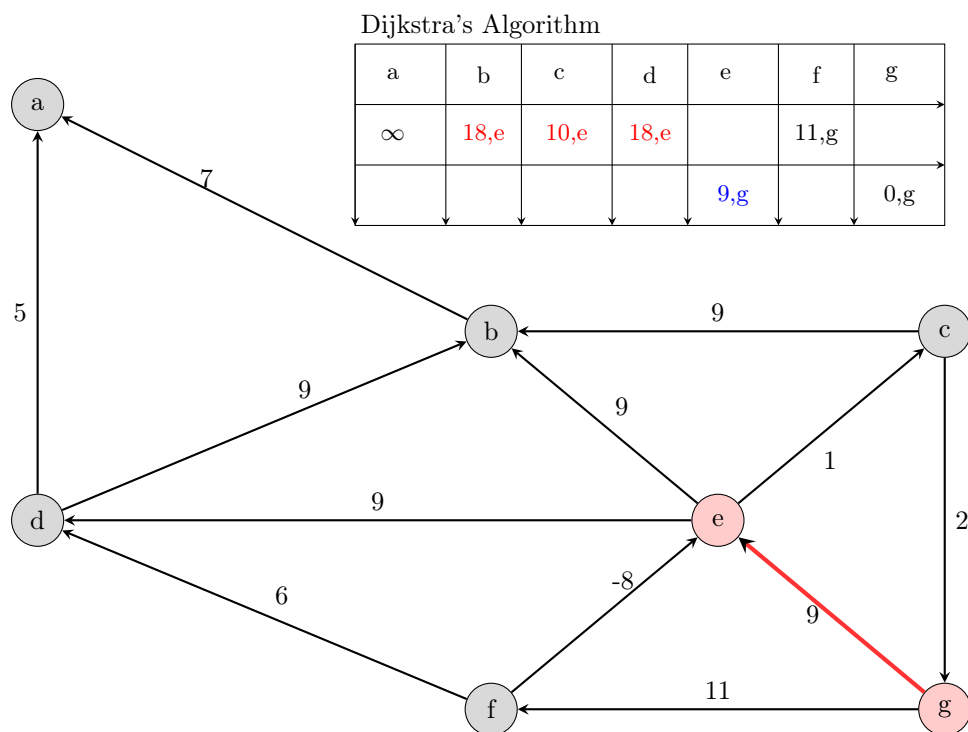


Figure 12: State after relaxing 'g-e'

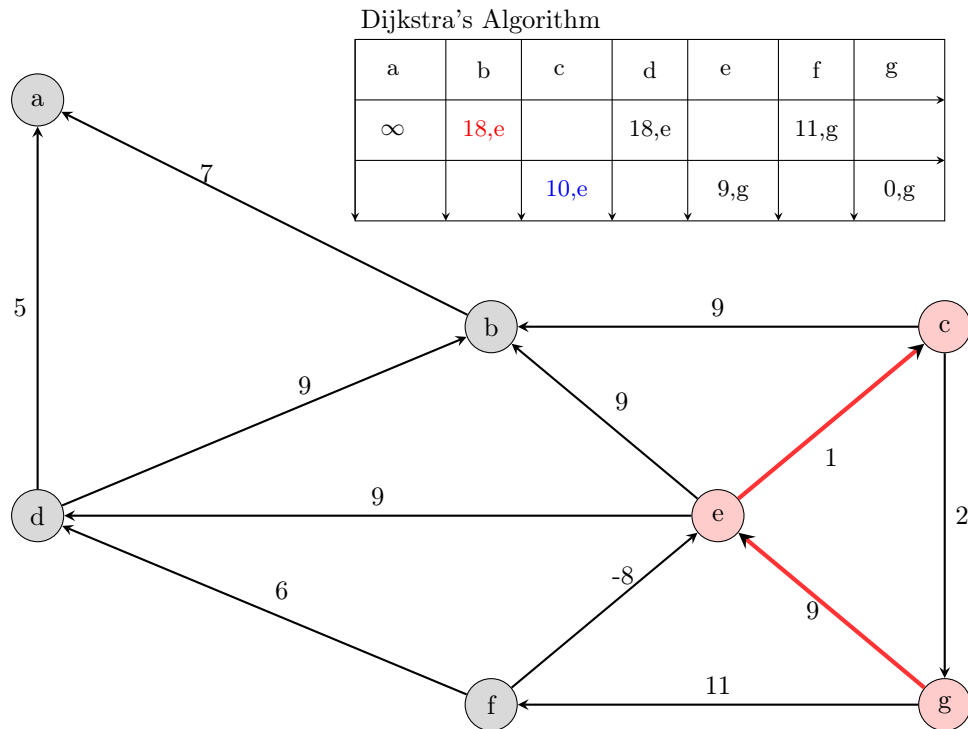


Figure 13: State after relaxing 'e-c'

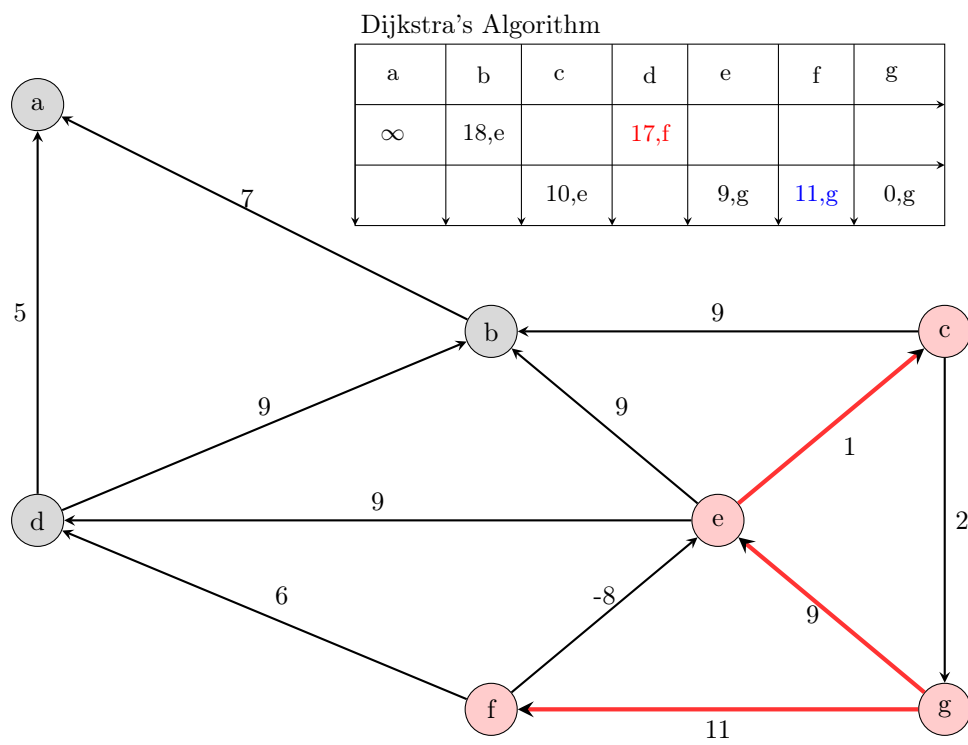


Figure 14: Relaxing neighbouring edges from 'f'

Dijkstra's Algorithm

a	b	c	d	e	f	g
22,d	18,e					
		10,e	17,f	9,g	11,g	0,g

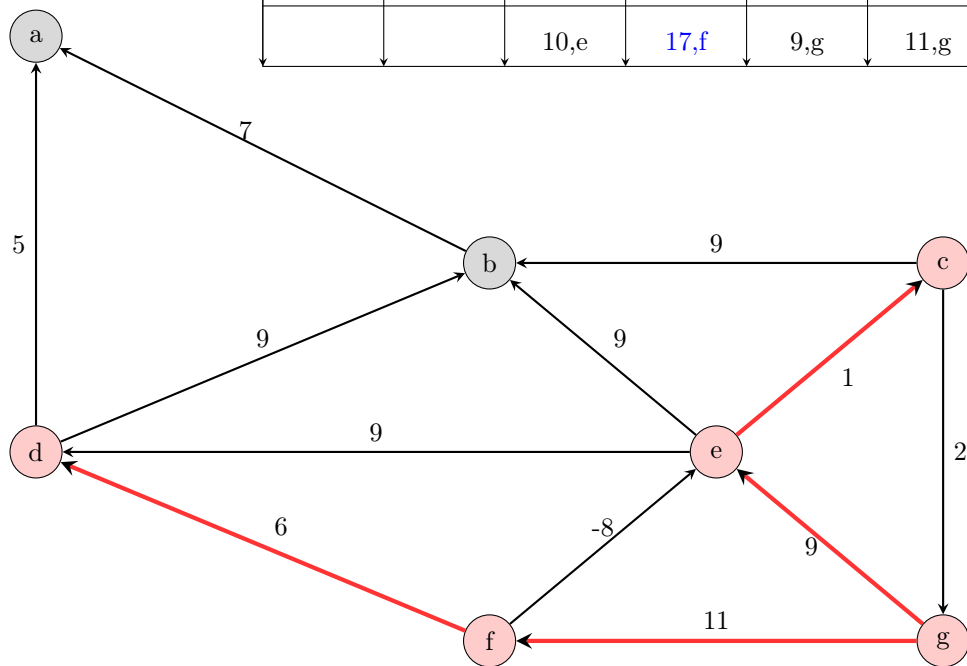


Figure 15: State with multiple finalized active paths

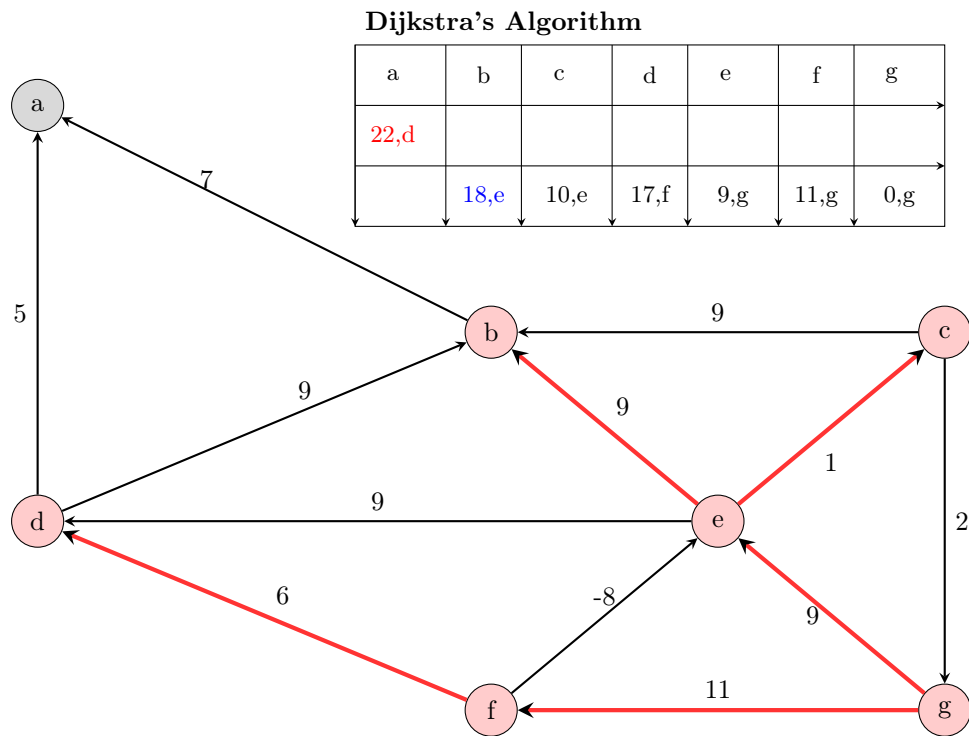


Figure 16: State with final path to 'a' activated

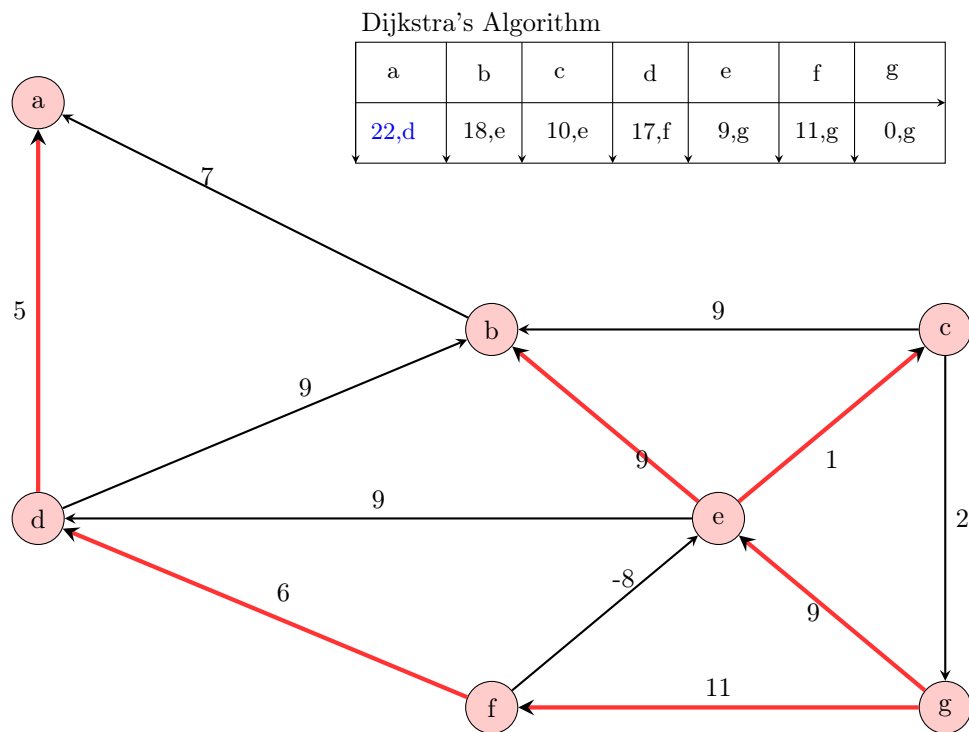


Figure 17: State with finalized path to 'a'

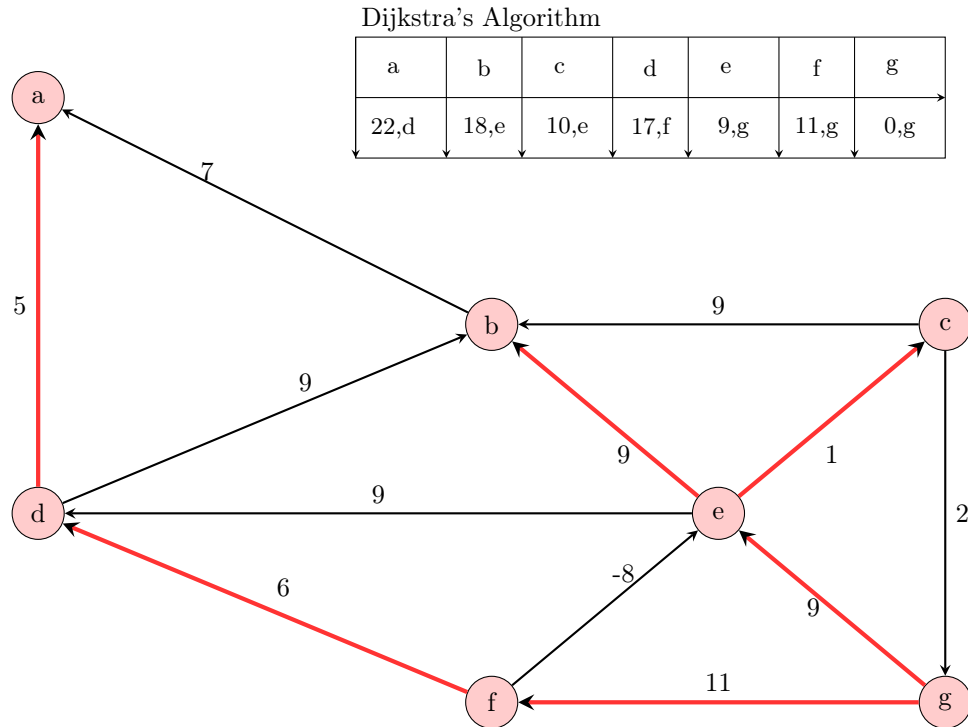


Figure 18: State with finalized path to 'a'

In this graph, it failed to account for the late-discovered reduction in path weight provided by the -8 edge from f to e.

Reason Why It fails:

Dijkstra's algorithm is built on a greedy strategy that assumes once a node is "visited," the shortest path to that node has been found and will never change. Negative edge weights break this fundamental assumption.

Dijkstra's algorithm maintains a set of reached nodes and always picks the one with the smallest tentative distance to "finalize." It assumes that because all edge weights are positive, adding any further edges to a path will only make that path longer.

Mathematically, if the shortest path to node u is $d(u)$, and we relax an edge (u, v) with weight w , the new distance $d(v) = d(u) + w$ is guaranteed to be $\geq d(u)$ if $w \geq 0$.

Solution:

If a graph contains negative edge weights but no negative cycles, the standard solution is the **Bellman-Ford Algorithm**.

How it works: Unlike Dijkstra's, which greedily "finalizes" a node, Bellman-Ford relaxes all $|V|-1$ edges in the graph repeatedly.

What if there exist a negative cycle

If a graph contains a negative cycle (a loop where the sum of the edge weights is less than zero), the concept of a "shortest path" actually breaks down.

In this scenario, Dijkstra's algorithm would likely get stuck in an infinite loop (if implemented with a priority queue that re-inserts updated nodes) or simply provide a wrong answer.

Here is why negative cycles are a "game over" for standard pathfinding:

The "Infinite Loop" Problem

If you have a cycle whose total weight is, for example, -5, every time you traverse that cycle, the total path weight decreases by 5.

Lap 1: Path weight = 10

Lap 2: Path weight = 5

Lap 3: Path weight = 0

Lap 100: Path weight = -485

Because you can keep circling forever to make the distance "smaller" (closer to $-\infty$), there is no mathematical "shortest" path.